

Tutorial: Pogotrack Data Analysis

Purpose. This document describes the analysis procedure used for Pogobot swarm experiments. It starts from the data folder produced during an experiment and covers detection, tracking, RGB identification, merging with robot measurements, and the final figures.

Maintainer. Keivan Amini

Date. 22 September 2026

Contact. For questions or improvements, contact amini@isir.upmc.fr or keivan.amini@espci.fr or open an issue or pull request in the repository.

Contents

1	Introduction	1
2	Installation	3
3	Repository and experiment structure	3
4	Core analysis	4
5	Important configuration parameters	5
6	Relevant modules and classes	6
7	Config-driven postprocessing	6
8	Understanding the merged data and slides	8
9	Troubleshooting checklist	8
10	Final remarks	9

1 Introduction

1.1 Aim and users

Pogotrack is a simple library used to analyse and reconstruct trajectories from videos of Pogobot swarms. The main output is a time series of robot positions and orientations, with a persistent identifier for each robot. The video data can then be joined with variables recorded by the robots, such as controller score and weight associated with each robot. The current maintainer is Keivan Amini, with contributions from

the experimental and robotics teams, including J  r  my Fersula. Most changes should be made through the YAML configuration or the postprocessing configuration; editing the detector itself should not be necessary for a standard experiment.

1.2 What the pipeline provides

- Detection of Pogobot contours and estimation of (x, y, θ) .
- Persistent trajectory linking with TrackPy.
- Phototaxis processing with arena masks, separate dark/light detection, cached masks, ROI detection, and fallback recovery.
- RGB-ID recordings in which the robot LEDs encode a ternary identifier.
- Exact-count detection and recovery for experiments with a fixed number of robots.
- Independent subprocess workers for analysing several experiments at the same time.
- Automatic estimates of the video-to-experiment time offset and RGB flashing start frame.
- Merging of video trajectories with Feather controller data.
- Summary figures showing controller response, population traces, density, and space-time fields.

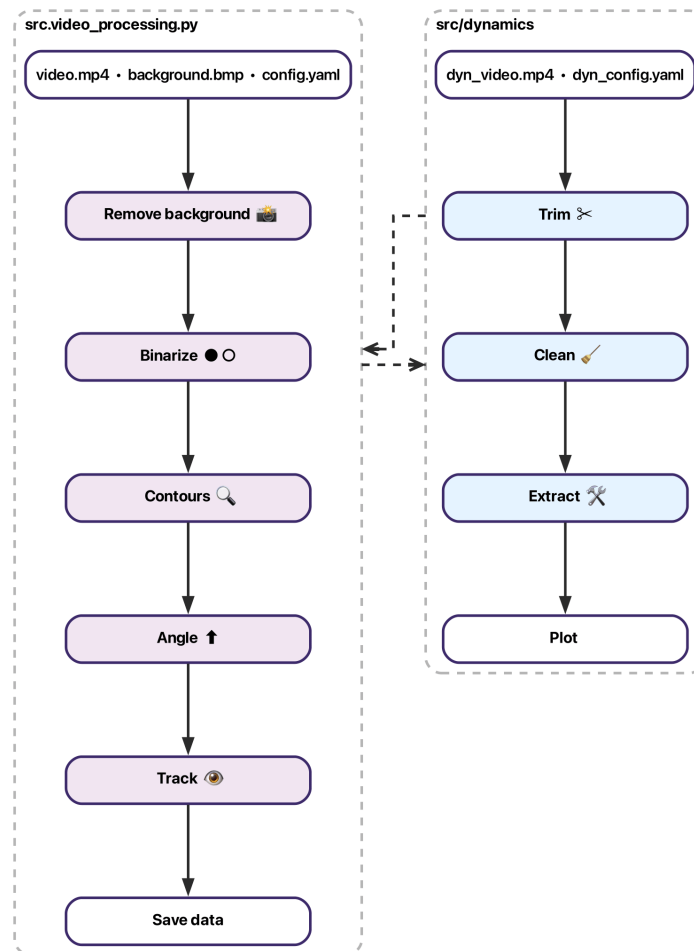


Figure 1: Overview of the Pogotrack analysis pipeline.

2 Installation

2.1 System requirements

Pogotrack supports CPython 3.11–3.13; Python 3.13 is recommended on Apple Silicon macOS. The versions used by the project are listed in `requirements.txt`. The dynamics workflow also needs the `ffmpeg` command-line program. The older `tmux` launcher needs `tmux` as well.

On a macOS computer using Homebrew, a typical installation is:

```
brew install python@3.13 ffmpeg tmux
cd /path/to/pogotrack
export DYLD_LIBRARY_PATH="$(brew --prefix expat)/lib${DYLD_LIBRARY_PATH:+:$DYLD_LIBRARY_PATH}"
python3.13 -m venv .venv
source .venv/bin/activate
python -m pip install -U pip
python -m pip install -r requirements.txt
```

On some Homebrew installations, the `DYLD_LIBRARY_PATH` line is needed because `ensurepip` selects the system Expat library. After activating the environment, check the interpreter before installing packages:

```
python --version
which python
```

The first command should report Python 3.11 or newer. The second should point inside `.venv/bin/`.

Attention. Do not run the pipeline with the old system interpreter `/usr/bin/python3`. Pogotrack now rejects Python versions older than 3.11. Always activate the project virtual environment first.

3 Repository and experiment structure

The most important repository directories are:

Path	Role
<code>main.py</code>	Command-line entry point for standard tracking, phototaxis tracking, RGB-ID tracking, and optional parallel frame processing.
<code>src/</code>	Core computer-vision, tracking, plotting, and dynamics modules.
<code>config/</code>	YAML configuration files. <code>default.yaml</code> is the principal tracking configuration.
<code>data/</code>	Experimental input folders containing videos, backgrounds, and robot measurements.
<code>results/</code>	Generated tracking CSV files, merged datasets, logs, plots, and slides.
<code>postprocessing/</code>	Batch analysis, RGB-ID decoding, automatic merging, and experiment-slide generation.
<code>stl/</code>	Pogobot exoskeleton files used by the experimental setup.
<code>requirements.txt</code>	Pinned Python dependencies for the supported runtime.

For one experiment, the recommended input layout is:

```
data/
+-- 18-09-26/
+-- alpha10-4_T100_v2.mp4      # normal tracking video
+-- id_alpha10-4_T100_v2.mp4   # RGB-ID video
```

```

+-- bkg.bmp                # background image
+-- alpha10-4_T100_v2.feather  # robot controller data

```

The two videos must belong to the same experiment. By default, the postprocessing configuration expects the RGB video to have `id_` added before the normal video name, the background to be called `bkg.bmp`, and the Feather file to have the experiment name. These names can be changed in `postprocessing/pipeline.yaml`.

Important. Keep the files for one experiment in the same date folder. The postprocessing script uses this layout to build paths and to avoid pairing a video with the wrong background or controller data.

4 Core analysis

4.1 Launching standard tracking

The main command analyses one video and writes the trajectory CSV given by `-output`:

```

source .venv/bin/activate

python -m main \
  --video data/18-09-26/alpha10-4_T100_v2.mp4 \
  --background data/18-09-26/bkg.bmp \
  --output results/18-09-26/alpha10-4_T100_v2/alpha10-4_T100_v2.csv \
  --config config/default.yaml

```

For a phototaxis experiment, the default configuration uses the dark/light arena split and the ROI-based detection path. The standard CSV contains:

```
time, x, y, theta, particle
```

Here `particle` is TrackPy's temporary identity. It is not the physical robot ID; that mapping is made later using the RGB-ID recording.

4.2 Parallel processing of one video

For normal phototaxis tracking, independent frame chunks can be processed by multiple workers:

```

python -m main \
  --video data/18-09-26/alpha10-4_T100_v2.mp4 \
  --background data/18-09-26/bkg.bmp \
  --output results/18-09-26/alpha10-4_T100_v2/alpha10-4_T100_v2.csv \
  --config config/default.yaml \
  --workers 2 \
  --warmup-frames 50

```

The video is split into contiguous frame ranges. Each worker receives context frames before its range so that local detection state can be initialized. The results are put back in frame order before TrackPy linking. This mode is for standard phototaxis tracking only; it cannot be combined with RGB-ID analysis or frame visualization.

4.3 RGB-ID tracking

The RGB-ID video is processed with `RGB_ID_ANALYSIS: true`. In normal use, `run_pipeline.py` creates a temporary configuration, sets the expected robot count, and launches this step. A direct command looks like this:

```
python -m main \\  
  --video data/18-09-26/id_alpha10-4_T100_v2.mp4 \\  
  --background data/18-09-26/bkg.bmp \\  
  --output results/18-09-26/alpha10-4_T100_v2/RGB_ID.csv \\  
  --config config/rgb_id_experiment.yaml
```

The RGB-ID CSV contains:

```
frame, time, id, x, y, theta, led_x, led_y, R, G, B
```

The RGB values are aggregated over the LED-flashing windows and decoded into ternary symbols during postprocessing.

5 Important configuration parameters

The main tracking settings are in `config/default.yaml`. The parameters below are the ones most likely to change when the arena or camera setup changes.

Parameter	Meaning
<code>N_POGO</code>	Expected number of robots. This is a central consistency constraint for detection and recovery.
<code>FPS</code>	Video frame rate used to convert frame numbers into seconds.
<code>THRESHOLD</code>	Foreground binarization threshold for non-phototaxis detection.
<code>AREAS</code>	Minimum and maximum accepted contour area in pixels squared.
<code>PERIMETERS</code>	Minimum and maximum accepted contour perimeter in pixels.
<code>CENTER, RADIUS</code>	Circular arena mask centre and radius in pixels.
<code>SEARCH_RANGE</code>	Maximum TrackPy linking distance in pixels.
<code>MEMORY</code>	Number of frames for which a temporarily missing robot can retain its TrackPy identity.
<code>POGOBOT_DIAMETER_CM, PIXEL_DIAMETER</code>	Pixel-to-centimetre calibration values.
<code>PHOTOTAXIS_ANALYSIS</code>	Enables the dark/light split processing path.
<code>PHOTOTAXIS_FAST</code>	Enables cached masks and ROI-based detection after initialization.
<code>PHOTOTAXIS_THRESHOLDS</code>	Dark and light segmentation thresholds.
<code>HOUGH_CIRCLES</code>	Circle-detection parameters used to identify robot bodies.
<code>RGB_ID_ANALYSIS</code>	Selects RGB-ID processing instead of standard trajectory processing.

In practice, start with the arena geometry, robot count, foreground thresholds, phototaxis thresholds, and calibration values. Change these in YAML rather than editing the detection code.

Attention. If the number of robots is fixed for an experiment, set `N_POGO` to that number. The detector uses it when checking and recovering incomplete frames.

6 Relevant modules and classes

6.1 Core tracking

- `main.py`: parses the command line, constructs `VideoProcessor`, and selects standard, parallel, or RGB-ID processing.
- `video_processing.py` in `src/`: contains the `VideoProcessor` class. Its principal methods are `process()`, `process_parallel()`, and `process_rgb_id()`.
- `src/Utils.py`: contains image preprocessing, contour and circle detection, pose estimation, pixel conversion, and tracking utilities.
- `src/plot_helpers.py`: contains frame debugging, trajectory plotting, RGB-ID debug views, and GIF/contact-sheet helpers.

The standard path subtracts the background, applies the arena mask, detects robot candidates, estimates their orientation, and stores one row per robot and frame. `TrackPy` then links the temporary particles over time.

6.2 Postprocessing modules

- `postprocessing/pipeline.yaml`: editable experiment list, paths, robot counts, merge settings, and slide settings.
- `postprocessing/run_pipeline.py`: runs tracking, merging, and slide generation from that configuration.
- `postprocessing/process_id_trits.py`: detects the RGB flashing start, aggregates RGB windows, decodes ternary IDs, assigns real robot IDs, and merges controller data.
- `postprocessing/experiment_slides_generator.py`: creates PNG and PDF summary figures for step, Gaussian, and tanh controllers.
- `postprocessing/batch_analysis_tmux.sh`: optional interactive tmux launcher for independent experiment workers.

7 Config-driven postprocessing

7.1 Configure an experiment

Edit `postprocessing/pipeline.yaml` before starting a new analysis. An entry gives the date folder, normal video, robot count, and any file-name overrides:

```
experiments:
- id: alpha10-4_T100_v2
  date: 18-09-26
  video: alpha10-4_T100_v2.mp4
  n_robots: 64
  probe_file: alpha10-4_T100_v2.feather
  outputs:
    normal: "{id}.csv"
    rgb_id: "RGB_ID.csv"
    merged: "merged_{id}.csv"
    slides: "{id}"
```

```
merge:
  seconds_difference: auto
  start_rgb_frame: auto
slides:
  enabled: true
  controller: tanh
```

With auto, the merge step checks the first ten seconds of the standard CSV for the first sustained population movement. It also searches the RGB-ID CSV for the LED onset. Replace either value with an integer if the automatic estimate is not suitable for a particular experiment.

7.2 Run all stages

From the repository root, with the virtual environment active:

```
python postprocessing/run_pipeline.py
```

The script runs the stages in this order:

1. standard and RGB-ID tracking for each experiment;
2. RGB-ID decoding and merging with the Feather data;
3. PNG and PDF slide generation.

Different experiments run concurrently as independent subprocesses. Set the number of concurrent tracking processes with `batch.max_workers`. A temporary tracking configuration is created for each experiment, so `config/default.yaml` is left unchanged.

7.3 Run one stage or one experiment

```
# Tracking only
python postprocessing/run_pipeline.py --stage batch

# Merge existing tracking outputs
python postprocessing/run_pipeline.py --stage merge

# Generate slides from existing merged outputs
python postprocessing/run_pipeline.py --stage slides

# Restrict any stage to one configured experiment
python postprocessing/run_pipeline.py \
  --stage merge \
  --experiment alpha10-4_T100_v2

# Inspect commands without executing them
python postprocessing/run_pipeline.py --dry-run
```

The output for one experiment is stored under:

```
results/<date>/<experiment>/
+-- <experiment>.csv
```

```
+-- RGB_ID.csv
+-- merged_<experiment>.csv
+-- <experiment>.png
+-- <experiment>.pdf
+-- tracking.log
+-- rgb_tracking.log
+-- merge.log
+-- slides.log
```

8 Understanding the merged data and slides

The merged dataset contains the main quantities used for swarm analysis:

`t, x, y, theta, q, w, id`

Here `t` is time after the detected offset, `x` and `y` are robot coordinates, `theta` is orientation, `q` and `w` are controller variables from the Feather file, and `id` is the decoded robot identifier.

The slide generator includes:

- the controller response panel;
- representative video frames at the initial, middle, and final times;
- the fraction of robots in the light region;
- per-robot and mean score/weight traces;
- relative density, weight, and score space-time fields.

The controller type is selected in the postprocessing configuration:

```
slides:
  enabled: true
  controller: tanh      # step, gaussian, or tanh
```

Additional slide options, such as light threshold, communication time, snapshot times, bin counts, and controller parameters, can be supplied under `slides.options`.

9 Troubleshooting checklist

If an analysis fails or the result looks wrong, check the following:

1. Confirm that the active interpreter is the project virtual environment and Python is at least 3.11.
2. Confirm that the normal video, `id_` RGB video, background image, and Feather file belong to the same experiment.
3. Confirm that `N_POGO` or `n_robots` matches the number of robots actually recorded.
4. Check arena centre, radius, foreground thresholds, and phototaxis dark/light thresholds.
5. Inspect `tracking.log` and `rgb_tracking.log` before changing algorithm code.

6. Run `-dry-run` to verify all generated paths and commands.
7. If automatic timing detection is unsuitable for one experiment, replace one or both `auto` values with manual values in `pipeline.yaml`.

Important. Keep the raw input data unchanged. Write generated CSV files, logs, and figures under `results/`; keep reusable settings in YAML files.

10 Final remarks

Pogotrack is a shared research tool and will continue to change as new experiments are added! If you find a bug or improve the analysis, please open an issue or submit a pull request.